

ANVAY Platform

Technical Architecture & Implementation Guide

Prepared for Project Review

Lead Engineer: Vitesh

1. Executive Summary & Architecture

The Elevator Pitch: ANVAY is a secure, multi-tenant event management platform. It features a decoupled architecture with a reactive **Angular** frontend, a robust **Spring Boot (Java 21)** REST API backend, and a **MySQL** relational database. The system is designed to handle multiple institutions, utilizing **JWT-based stateless authentication** for secure access across roles like Super Admins, Club Leaders, and Students.

2. The Complete Application Flow (Request Lifecycle)

This details the journey of a single piece of data (e.g., a student requesting the Event Feed) from the browser to the database and back.

Step 1: The Frontend Trigger (Angular)

When the user opens the application, Angular loads the respective component (e.g., student-component). The Angular service fires an HTTP GET request to fetch the events. Before the request leaves the browser, an Angular **HTTP Interceptor** catches it and attaches the user's JWT (JSON Web Token) to the Authorization header as a "Bearer" token.

Step 2: The Network Router

Because the frontend runs on port 4200 and the backend on port 8081, cross-origin restrictions apply. We solved this by configuring a `proxy.conf.json` in Angular and binding the host to `0.0.0.0`. This safely proxies the request from the frontend domain directly to the backend API, bypassing CORS issues during development.

Step 3: The Security Gatekeeper (Spring Security)

The request hits the Spring Boot application on port 8081. Before reaching any business logic, it is intercepted by the `SecurityFilterChain`. It hits our custom `JwtAuthenticationFilter`, which extracts the token, verifies the secret key, and checks expiration. If valid, Spring Security establishes a secure `SecurityContext` containing the user's roles.

Step 4: The Traffic Cop (Controllers)

The request is routed to the `EventController`. The Controller's sole responsibility is validating incoming URL parameters and passing the request to the Service layer.

Step 5: The Brains (Service Layer)

The `EventServiceImpl` contains the core business logic. It requests the active events from the database. A crucial engineering decision made here is the use of **Data Transfer Objects (DTOs)**. Instead of sending the heavy Event Entity (which contains massive recursive lists), the Service maps the data into a slim `EventFeedDTO` and calculates dynamic fields (like `isRegistered`).

Step 6: Data Access (Hibernate/JPA + MySQL)

The Service calls the `EventRepository`. Hibernate translates the Java method into a native SQL query, fetching data from the MySQL database. The data flows back up to the Controller, which wraps the DTO in an HTTP 200 OK response.

Step 7: The UI Render (Angular)

The Angular component receives the JSON array and utilizes structural directives (like `*ngFor` or `@for`) to dynamically render the Event Cards on the screen.

3. Deep Dive: JSON Web Tokens (JWT)

The Analogy: Think of a JWT like a VIP wristband at a music festival. You show your ID (username/password) at the ticket booth, and they give you a wristband. From then on, security guards just check your wristband—you don't need to show your ID again.

Token Architecture

A JWT consists of three Base64-encoded parts separated by periods (xxxxx . yyyy . zzzzz):

- **The Header (xxxxx):** Specifies the token type (JWT) and the signing algorithm (e.g., HS256).
- **The Payload (yyyyy):** Contains *Claims* (statements about the user). This includes registered claims like `exp` (expiration) and custom claims like the user's email. *Note: The payload is encoded, not encrypted, so sensitive data is never stored here.*
- **The Signature (zzzzz):** Created by hashing the Header, Payload, and the server's `app.jwt.secret`. If a malicious user alters their role in the payload, the signature is invalidated, and the server rejects it.

Code-Level Implementation

Backend (Spring Boot)

- **application.properties:** Defines the secret key and expiration time.
- **JwtUtils.java:** Contains the minting logic. Uses JJWT to build the token, setting the Subject, Expiration, and hashing it with the secret key upon successful login.
- **JwtAuthenticationFilter.java:** A `OncePerRequestFilter` that intercepts all traffic. It extracts the token from the `Authorization` header (stripping the "Bearer " prefix), validates the signature via `JwtUtils`, and if valid, triggers the authentication process.

Frontend (Angular)

- **Login Component:** Receives the token from the backend and stores it in `localStorage`.

- **AuthInterceptor.ts:** Implements `HttpInterceptor`. It catches every outgoing HTTP request, retrieves the token from memory, clones the request, and injects the `Authorization` header automatically.
- **AuthGuard.ts:** Implements `CanActivate` to protect UI routes, redirecting unauthenticated users to the login page.

Why JWT? (Engineering Justification)

We chose JWT because it is **stateless**. Traditional sessions require the server to store session IDs in memory, tying up resources. With JWT, the backend remembers nothing; the token proves the identity mathematically. This architecture is highly scalable and elegantly bypasses complex Cross-Origin (CORS) session issues.

4. Deep Dive: Spring Security & RBAC

This section outlines how we bridge the gap between receiving a JWT and establishing internal application access rights.

The Security Context & Thread-Local Storage

When the `JwtAuthenticationFilter` validates a token, it executes the following core logic:

```
String email = jwtUtil.extractEmail(token);
UserDetails userDetails = userDetailsService.loadUserByUsername(email);
UsernamePasswordAuthenticationToken authToken =
    new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
SecurityContextHolder.getContext().setAuthentication(authToken);
```

Step-by-Step Breakdown:

1. **Extraction:** The email is pulled from the trusted token.
2. **Loading Profile:** `CustomUserDetailsService` fetches the user from the database and maps them to a Spring `UserDetails` object.
3. **The ID Badge:** We create a `UsernamePasswordAuthenticationToken`. We pass the principal (the user details), `null` for credentials (we don't need the password since the JWT is valid), and the mapped authorities.
4. **The Lanyard (SecurityContext):** We place the token into the `SecurityContextHolder`. This utilizes **Thread-Local Storage**, guaranteeing that User A's identity is securely glued exclusively to User A's specific HTTP request thread. Once the response is sent, the thread dies and the context is wiped clean, maintaining true statelessness.

Role-Based Access Control (RBAC)

We did not create complex, tangled tables for permissions. RBAC was implemented natively using Spring's method security.

- **Database Level:** Roles are stored as simple strings/enums on the User entity (e.g., "SUPER_ADMIN").
- **The Mapping:** Inside CustomUserDetailsService, we map the database role by prepending Spring's required prefix: `new SimpleGrantedAuthority("ROLE_" + user.getRole().toUpperCase())`.
- **Enforcement:** We use `@PreAuthorize("hasRole('SUPER_ADMIN')")` on Controller endpoints. If a Student tries to hit this endpoint, Spring intercepts it and returns a 403 Forbidden error before the logic executes.

5. Key Engineering Highlights

The Data Initializer (Bootstrapping)

To solve the "first-run lockout" problem, we implemented a `CommandLineRunner` (`DataInitializer`). On application startup, it checks if the `admin@anvay.com` user exists. If not, it automatically securely hashes the configured properties password, injects the Super Admin, and assigns the required roles.

DTOs & Infinite Recursion

In relational databases (JPA/Hibernate), Entities are heavily linked. If we serialized native Entities directly to JSON, bidirectional relationships (e.g., Users to Clubs to Users) would cause an infinite recursion loop, crashing the server. We engineered a robust **DTO (Data Transfer Object)** layer to flatten the data, which shrinks payload sizes, saves bandwidth, and strictly controls what data is exposed to the client.

6. Codespaces Deployment Playbook

This is the exact sequence of commands and configurations utilized to deploy the decoupled full-stack environment within GitHub Codespaces, simulating a production Linux container network.

Phase 1: Environment Configuration

```
# Install required Java version (Java 21)
sdk install java 21.0.2-ms
sdk default java 21.0.2-ms

# Fix Maven permissions
cd anvay/anvay
chmod +x mvnw
```

Phase 2: Database Spin-up & Security Bypass

Ubuntu's default MySQL installation blocks external network JDBC logins for the root user via socket plugins. We bypassed this to allow Spring Boot to connect.

```
sudo service mysql start
sudo mysql -u root

# Inside MySQL Monitor:
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'root';
FLUSH PRIVILEGES;
CREATE DATABASE IF NOT EXISTS anvay;
exit;
```

Phase 3: Backend Deployment

Updated `application.properties` with the new root password, then forced a clean compilation to clear cached binaries.

```
# Run the backend
./mvnw clean spring-boot:run

# Note: If port 8081 is blocked by a zombie process, clear it:
fuser -k 8081/tcp
```

Phase 4: Frontend Routing Configuration

Angular's default dev server is isolated to localhost. To expose it to the Codespaces cloud router, we modified package.json and forced a port change to bypass aggressive GitHub caching causing 404 errors.

```
# package.json modification:
"start": "ng serve --host 0.0.0.0 --proxy-config proxy.conf.json"

# In a new terminal tab:
cd frontend/anvay-app
npm start

# If 404 persists, force a new tunnel on a new port:
npx ng serve --host 0.0.0.0 --port 4201 --proxy-config proxy.conf.json
```

Phase 5: Port Forwarding & Security Walls

By default, GitHub Codespaces blocks external traffic with an authentication wall. We navigated to the **Ports** tab in VS Code and set the visibility for **both** the frontend port and backend port (8081) to **Public**. Making the backend public is critical—without it, the frontend UI would load, but all subsequent API requests would fail CORS/Authentication checks.